

JAVASCRIPT

1. Introduction to JavaScript

- **What is JavaScript?**
 - History and evolution
 - Client-side vs. server-side scripting
- **Setting up your environment**
 - Browsers and Developer Tools
 - Installing a code editor (e.g., VS Code)
 - Basic debugging in Developer Tools

2. JavaScript Basics

- **Syntax and Fundamentals**
 - Variables (`var`, `let`, `const`)
 - Data types (string, number, boolean, null, undefined, symbol, bigint)
 - Operators (arithmetic, assignment, comparison, logical)
- **Control Structures**
 - Conditionals (`if`, `else`, `else if`, `switch`)
 - Loops (`for`, `while`, `do...while`, `for...of`, `for...in`)
- **Functions**
 - Function declarations and expressions
 - Arrow functions
 - Parameters and return values
 - Scope and closures

3. DOM Manipulation

- **Document Object Model (DOM)**
 - Selecting elements (`getElementById`, `querySelector`, etc.)
 - Modifying HTML and CSS (`innerHTML`, `style`, `classList`)
 - Event handling (`addEventListener`, event types)
- **Forms and Input**
 - Form validation
 - Event listeners for forms
 - Dynamic form inputs

4. Intermediate Concepts

- **Arrays and Objects**
 - Creating and manipulating arrays
 - Array methods (`map`, `filter`, `reduce`, `forEach`)
 - Working with objects and object methods
- **Error Handling**
 - `try...catch`
 - `throw` statements
 - Custom error messages
- **Asynchronous JavaScript**
 - Callbacks
 - Promises (`then`, `catch`, `finally`)
 - `async/await`

5. Advanced Topics

- **ES6+ Features**
 - Destructuring
 - Template literals
 - Default parameters
 - Rest and spread operators
 - Modules (`import` and `export`)
- **Object-Oriented Programming (OOP)**
 - Prototypes and inheritance
 - Classes and constructors
 - Encapsulation, inheritance, and polymorphism
- **Functional Programming**
 - Pure functions
 - Higher-order functions
 - Immutability
 - Currying and partial application

6. Working with APIs

- **AJAX and Fetch API**
 - Making HTTP requests
 - Consuming APIs
 - Error handling in API calls
- **JSON**
 - Parsing and stringifying JSON data
 - Working with API responses

7. Browser-Specific Features

- **LocalStorage, SessionStorage, and Cookies**
- **Geolocation API**
- **Web Workers**
- **Canvas and SVG**

8. Testing and Debugging

- **Debugging Techniques**
 - Using breakpoints
 - The `console` object (`log`, `warn`, `error`, `table`)
- **Testing Frameworks**
 - Unit testing with Jest or Mocha
 - Writing test cases

9. JavaScript Build Tools

- **Introduction to npm and Yarn**
- **Module bundlers**
 - Webpack
 - Parcel
- **Transpilers**
 - Babel

10. Frameworks and Libraries

- **Introduction to Popular Frameworks**
 - React.js
 - Vue.js
 - Angular
- **Utility Libraries**
 - Lodash
 - Moment.js (or alternatives like Day.js)

11. Real-World Applications

- **Building Projects**
 - To-Do List
 - Weather App (using an API)
 - Portfolio Website
 - E-commerce Cart
 - Real-time Chat App
- **Version Control**
 - Git basics

- Hosting projects on GitHub

12. Performance Optimization

- **Optimizing JavaScript**
 - Minification and bundling
 - Lazy loading
 - Debouncing and throttling
- **Web Performance APIs**
 - Timing APIs
 - Analyzing performance in DevTools

13. Advanced Topics (Optional)

- **TypeScript**
 - Introduction and usage with JavaScript
- **Node.js**
 - Setting up a backend
 - Creating REST APIs
- **WebAssembly**
 - Introduction and use cases

REACT.JS (Frontend combo-javascript+react.js)

Prerequisites

Before diving into React, ensure you have a good understanding of:

1. **HTML**: Semantic tags, forms, and basic structure.
2. **CSS**: Flexbox, Grid, responsive design.
3. **JavaScript Basics**: ES6 features, functions, arrays, objects, DOM manipulation, and `fetch`.
4. **Version Control**: Basic Git and GitHub knowledge.

1. Introduction to React

- **What is React?**

- Overview of React and its ecosystem.
- Why React? (Advantages and use cases)
- Difference between SPA and traditional applications.
- **Setting Up the Environment**
 - Installing Node.js and npm.
 - Setting up a React project using `create-react-app` or Vite.
 - Exploring the folder structure.

2. React Basics

- **React Components**
 - Functional components and JSX.
 - Props and state (difference and use cases).
- **JSX Syntax**
 - Embedding expressions.
 - Conditional rendering.
 - Lists and keys.
- **Event Handling**
 - Handling user input with event listeners.
 - Passing arguments to event handlers.

3. State and Lifecycle

- **State Management**
 - Using `useState` for state in functional components.
 - Updating and accessing state.
- **Lifecycle Methods**
 - Understanding React component lifecycle.
 - Using `useEffect` for side effects.
 - Dependency arrays and cleanup functions.

4. Handling Forms

- **Controlled Components**
 - Handling form inputs using state.
 - Two-way data binding.
- **Uncontrolled Components**
 - Working with `ref` for direct DOM manipulation.
- **Form Validation**

- Basic validation logic.
- Integrating with libraries like Formik or React Hook Form.

5. Advanced React Concepts

- **React Router**
 - Setting up routes with `react-router-dom`.
 - Nested routes, redirects, and navigation.
 - Using `useParams`, `useNavigate`, and route guards.
- **Context API**
 - Sharing state across components.
 - Creating and using context providers and consumers.
 - Avoiding prop drilling.
- **Custom Hooks**
 - Writing reusable custom hooks.
 - Examples: Fetching data, managing forms, debouncing.

6. State Management

- **Lifting State Up**
 - Sharing state between sibling components.
- **Redux (or Alternatives like Zustand, Recoil, Jotai)**
 - Introduction to Redux: Store, actions, reducers.
 - Using `redux-toolkit` for simpler state management.
 - Middleware like Thunk or Saga.

7. APIs and Data Fetching

- **Fetching Data**
 - Using `fetch` and `axios` for API calls.
 - Displaying loading and error states.
- **React Query**
 - Simplifying data fetching with React Query.
 - Caching, background fetching, and mutations.

8. Performance Optimization

- **React Performance**
 - Understanding rendering and reconciliation.
 - Using `React.memo` and `useCallback` for optimization.

- **Code Splitting**
 - Lazy loading components with `React.lazy` and `Suspense`.
- **Error Boundaries**
 - Handling errors in React components.

9. Testing in React

- **Unit Testing**
 - Writing tests with Jest.
 - Testing components with React Testing Library.
- **End-to-End Testing**
 - Setting up Cypress or Playwright.
 - Writing E2E tests for user flows.

10. Advanced Topics

- **TypeScript with React**
 - Adding TypeScript to a React project.
 - Using types and interfaces with props and state.
- **Server-Side Rendering (SSR)**
 - Using Next.js for SSR and static site generation.
- **Progressive Web Apps (PWA)**
 - Making React apps offline-ready.
- **React Native**
 - Extending React knowledge to mobile app development.

11. Building Real-World Projects

- **Basic Projects**
 - To-Do List
 - Calculator
 - Weather App
- **Intermediate Projects**
 - Movie Search App with API integration.
 - Blog CMS with CRUD functionality.
 - E-commerce Storefront.
- **Advanced Projects**
 - Chat Application with WebSockets.
 - Social Media Dashboard.
 - Real-Time Data Visualization Dashboard.

12. Deploying React Applications

- **Static Deployment**
 - Deploying to platforms like Netlify, Vercel, or GitHub Pages.
- **Full-Stack Deployment**
 - Integrating with a backend (Node.js, Django, etc.).
 - Deploying full-stack apps on Heroku or AWS.